

Project Title

Real-Time Weather Dashboard using Asynchronous JavaScript & REST APIs

Objective

The objective of this project is to build a professional real-time Weather Dashboard using asynchronous JavaScript and RESTful APIs. The application dynamically fetches live weather data from a public API and displays weather metrics such as temperature, wind speed, and weather conditions based on the city entered by the user.

Tools Used

- HTML5
- CSS3
- JavaScript
- Open-Meteo REST API
- Visual Studio Code / Notepad
- Google Chrome Browser

Dataset Description

This project uses live weather data fetched dynamically from the Open-Meteo REST API. The API returns JSON-formatted weather information including temperature, wind speed, latitude, longitude, and weather conditions.

Machine Learning Algorithms Used

No machine learning algorithms are used in this project because it is based on asynchronous JavaScript programming and REST API integration.

Key Features

- Fetch API with Async/Await

- Real-Time Weather Data
- Search Weather by City Name
- JSON Data Parsing
- Dynamic DOM Rendering
- Error Handling for Invalid Requests
- Responsive Professional UI
- Glassmorphism Weather Dashboard Design

Steps Performed

1. Created the HTML structure for the weather dashboard.
2. Designed a modern responsive UI using CSS3.
3. Integrated Open-Meteo REST API.
4. Used asynchronous JavaScript with async/await.
5. Parsed nested JSON weather data dynamically.
6. Implemented city search functionality.
7. Added error handling for invalid city names and network issues.
8. Dynamically displayed weather metrics on the webpage.
9. Tested responsiveness and application functionality.

Output

- Successfully developed a real-time weather dashboard application.
- Implemented asynchronous API integration using Fetch API.
- Displayed accurate live weather information dynamically.
- Added responsive and professional UI design.
- Enabled city-based weather search functionality.

Professional HTML, CSS & JavaScript Code

```
<!DOCTYPE html>
<html lang="en">
<head>

<meta charset="UTF-8">
```

```
<meta name="viewport"
content="width=device-width, initial-scale=1.0">

<title>Weather Dashboard</title>

<style>

*{
  margin:0;
  padding:0;
  box-sizing:border-box;
  font-family:'Segoe UI',sans-serif;
}

body{

  min-height:100vh;

  background:
  linear-gradient(135deg,#0f2027,#203a43,#2c5364);

  display:flex;

  justify-content:center;

  align-items:center;

  color:white;
}

/* Main Container */

.container{

  width:90%;
  max-width:900px;

  background:rgba(255,255,255,0.08);

  border-radius:25px;

  padding:40px;
```

```
        backdrop-filter:blur(10px);

        box-shadow:0 10px 30px rgba(0,0,0,0.4);
    }

    /* Header */

    .header{

        text-align:center;

        margin-bottom:30px;
    }

    .header h1{

        font-size:45px;

        color:#00d4ff;
    }

    .header p{

        margin-top:10px;

        color:#ddd;
    }

    /* Search Box */

    .search-box{

        display:flex;

        gap:15px;

        margin-top:20px;
    }

    .search-box input{
```

```
    flex:1;

    padding:15px;

    border:none;

    border-radius:10px;

    outline:none;

    font-size:16px;
}

.search-box button{

    padding:15px 25px;

    border:none;

    border-radius:10px;

    background:#00d4ff;

    color:black;

    font-weight:bold;

    cursor:pointer;

    transition:0.3s;
}

.search-box button:hover{

    background:white;
}

/* Weather Section */

.weather-container{

    margin-top:35px;
```

```
    display:none;

    grid-template-columns:repeat(2,1fr);

    gap:20px;
}

.card{

    background:rgba(255,255,255,0.12);

    padding:25px;

    border-radius:20px;

    text-align:center;
}

.card h2{

    margin-bottom:10px;

    color:#00d4ff;
}

.card p{

    font-size:28px;

    font-weight:bold;
}

/* Error Message */

#error{

    margin-top:20px;

    text-align:center;

    color:yellow;
```

```
        font-weight:bold;
    }

    /* Responsive */

    @media(max-width:768px){

        .weather-container{

            grid-template-columns:1fr;
        }

        .search-box{

            flex-direction:column;
        }

        .search-box button{

            width:100%;
        }

        .header h1{

            font-size:35px;
        }
    }

</style>

</head>

<body>

<div class="container">

    <div class="header">

        <h1>Weather Dashboard</h1>

        <p>
```

Real-Time Weather Application using
Async JavaScript & REST API

</p>

</div>

<div class="search-box">

<input type="text"
id="cityInput"
placeholder="Enter City Name">

<button onclick="getWeather()">
Search
</button>

</div>

<div id="error"></div>

<div class="weather-container"
id="weatherContainer">

<div class="card">

<h2>City</h2>

<p id="city"></p>

</div>

<div class="card">

<h2>Temperature</h2>

<p id="temperature"></p>

</div>

<div class="card">

<h2>Wind Speed</h2>


```

        <p id="wind"></p>

    </div>

    <div class="card">

        <h2>Weather Code</h2>

        <p id="condition"></p>

    </div>

</div>

<script>

async function getWeather(){

    const city =
    document.getElementById("cityInput").value;

    const error =
    document.getElementById("error");

    const container =
    document.getElementById("weatherContainer");

    error.innerHTML = "";

    try{

        /* Get Latitude & Longitude */

        const geoResponse =
        await fetch(
`https://geocoding-api.open-
meteo.com/v1/search?name=${city}&count=1`
        );

```

```

const geoData =
await geoResponse.json();

if(!geoData.results){

    throw new Error(
        "City not found"
    );
}

const lat =
geoData.results[0].latitude;

const lon =
geoData.results[0].longitude;

const cityName =
geoData.results[0].name;

/* Fetch Weather Data */

const weatherResponse =
await fetch(
`https://api.open-
meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current_weather=true`
);

const weatherData =
await weatherResponse.json();

const current =
weatherData.current_weather;

/* Display Data */

document.getElementById("city")
.innerHTML = cityName;

document.getElementById("temperature")
.innerHTML =
current.temperature + " °C";

```

```
        document.getElementById("wind")
        .innerHTML =
        current.windspeed + " km/h";

        document.getElementById("condition")
        .innerHTML =
        current.weathercode;

        container.style.display =
        "grid";
    }

    catch(err){

        container.style.display =
        "none";

        error.innerHTML =
        "Invalid City Name or Network Error";
    }

}

</script>

</body>
</html>
```

Conclusion

This project successfully demonstrates asynchronous JavaScript programming and RESTful API integration by creating a professional real-time Weather Dashboard application. The application dynamically fetches live weather data, processes JSON responses, and displays weather metrics in a responsive and modern user interface.

Expected Outcome

- Fully functional real-time weather dashboard
- Professional responsive UI design

- Dynamic weather search functionality
- Real-time weather metrics display
- Error handling for invalid requests
- Improved asynchronous JavaScript and API integration skills

OUTPUT

